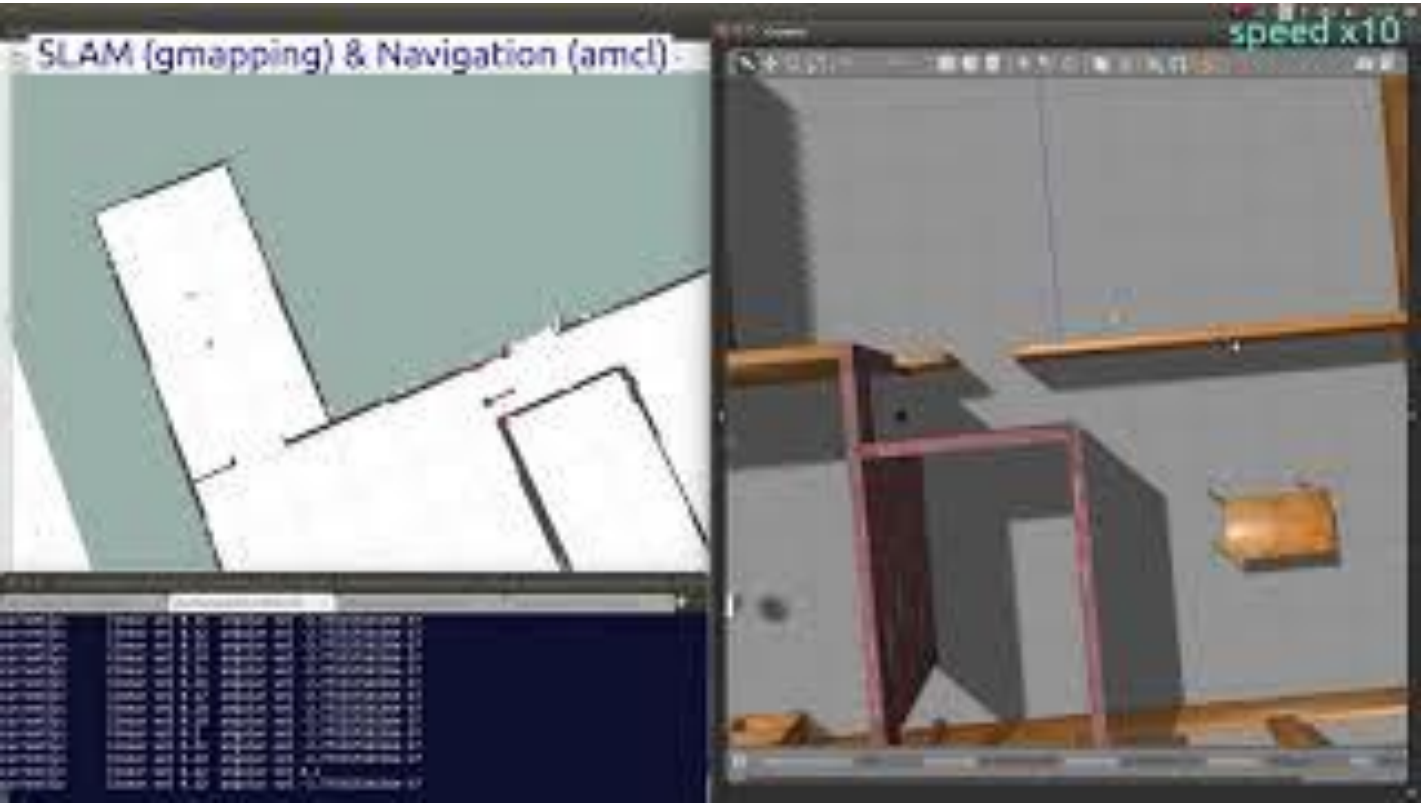
UNIVERSITAT JAUME I

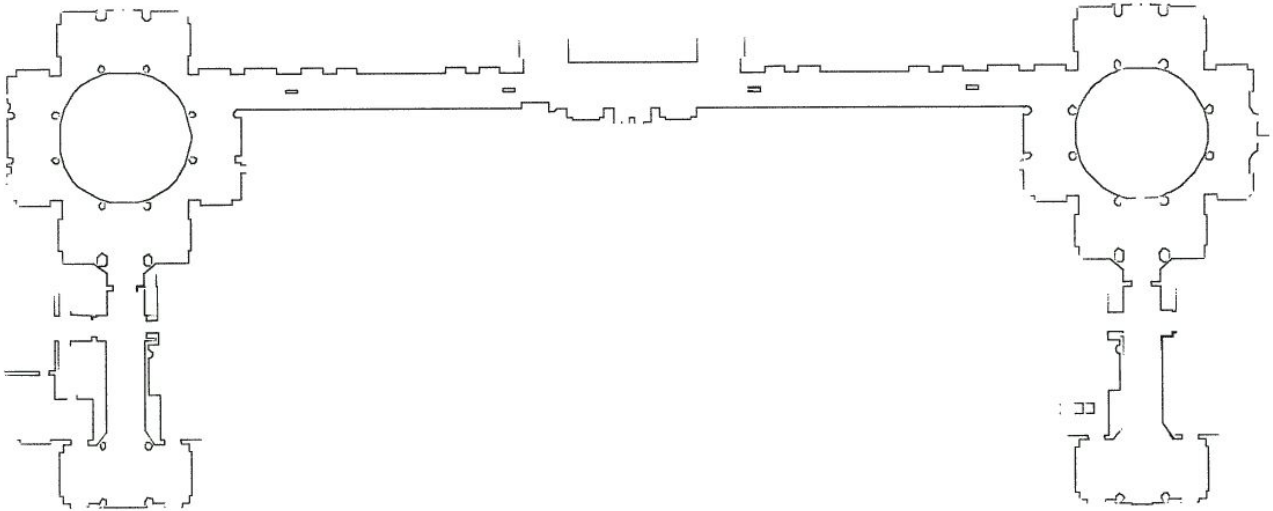
Mobile Robotics



Bachelor's Degree in Robotic Intelligence

SLAM: Simultaneous Localization and Mapping

?



File Panels Help

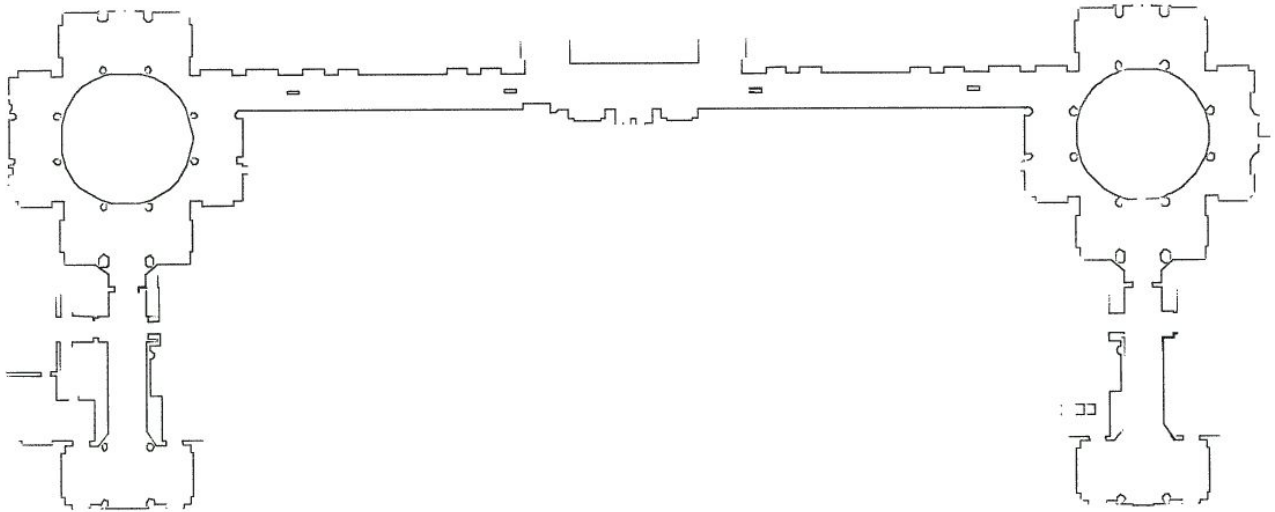
Interact | Move Camera | Select | Focus Camera | Measure | 2D Pose Estimate | 2D Goal Pose | Publish Point

Displays

- Global Options
  - Fixed Frame: map
  - Background Color: 255; 255; 255
  - Frame Rate: 30
- Global Status: Ok
  - Fixed Frame: OK
- Grid
- Map
- Odometry
  - Status: Ok
  - Topic: /odom
  - Position Tolerance: 0.1
  - Angle Tolerance: 0.1
  - Keep: 0
  - Shape: Arrow
  - Covariance
- LaserScan
  - Status: Ok
  - Topic: /scan
  - Selectable
  - Style: Flat Squares
  - Size (m): 0.1
  - Alpha: 1
  - Decay Time: 10000
  - Position Transfor...: XYZ

Background Color
Background color for the 3D view.

Add | Duplicate | Remove | Rename



Time

ROS Time: 1791359764.12 | ROS Elapsed: 66.31 | Wall Time: 1791359764.15 | Wall Elapsed: 66.31 | Experimental

Reset | 28 fps

SLAM

Given:

- a series of controls and
- sensor observations over discrete time steps,

the SLAM problem is to compute:

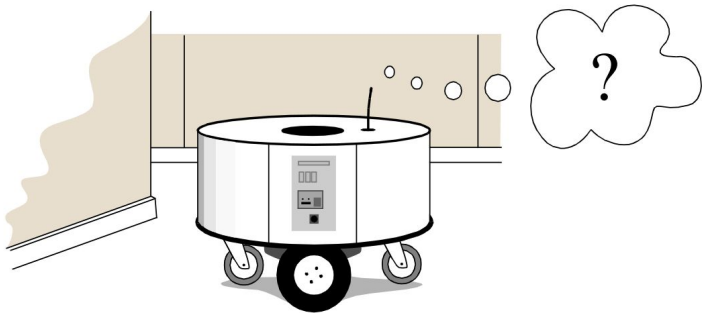
- an estimate of the agent's state and
- a map of the environment.

All quantities are usually probabilistic.

https://en.wikipedia.org/wiki/Simultaneous_localization_and_mapping

General schematic for concurrent localization and map building

[Siegwart et al., 2011]



Map

Map Building and Maintenance

Refine Feature Parameters
increase credibility

Add New Features
extend map

Remove Offensive Features
decrease credibility

Encoder

position estimate

Prediction of Measurement and Position (odometry)

Estimation(fusion)
using confirmed map

predicted feature observations

matched predictions and observations

unexpected observations

unobserved predictions

YES

YES

Matching

NO

Unexpected Observation?

NO

raw sensor data or extracted features

Perception

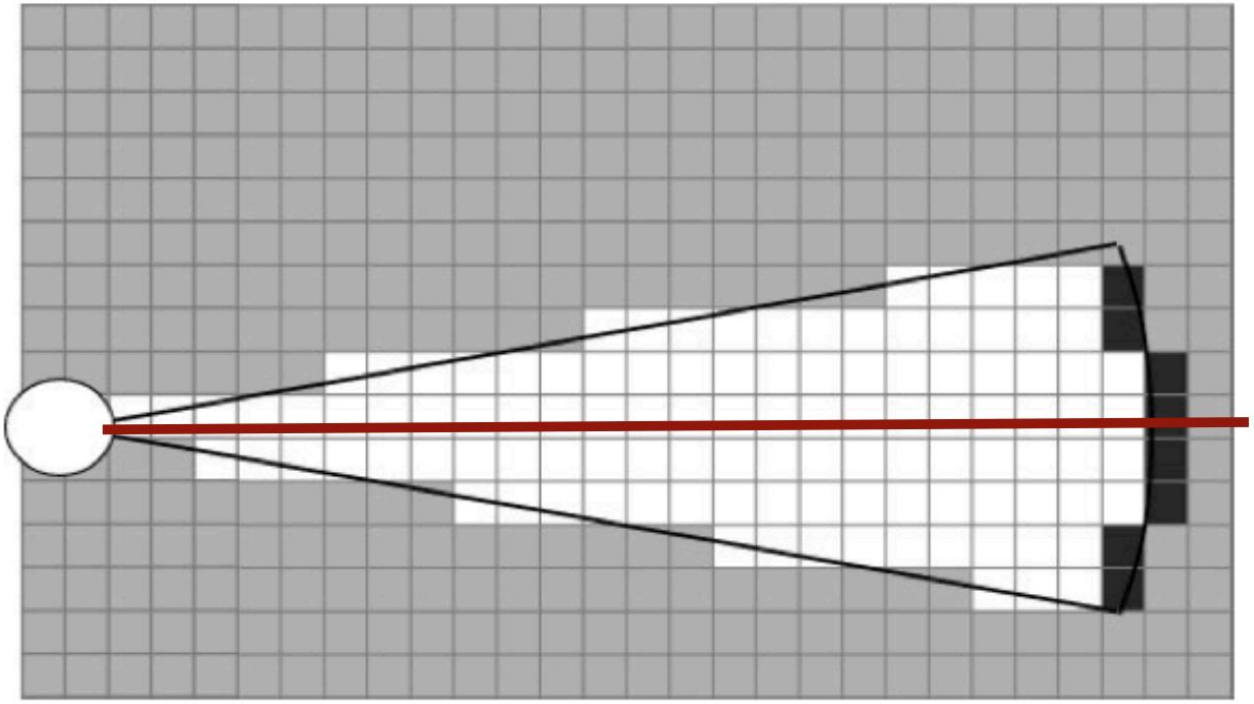
Observation
on-board sensors

Occupancy grid map

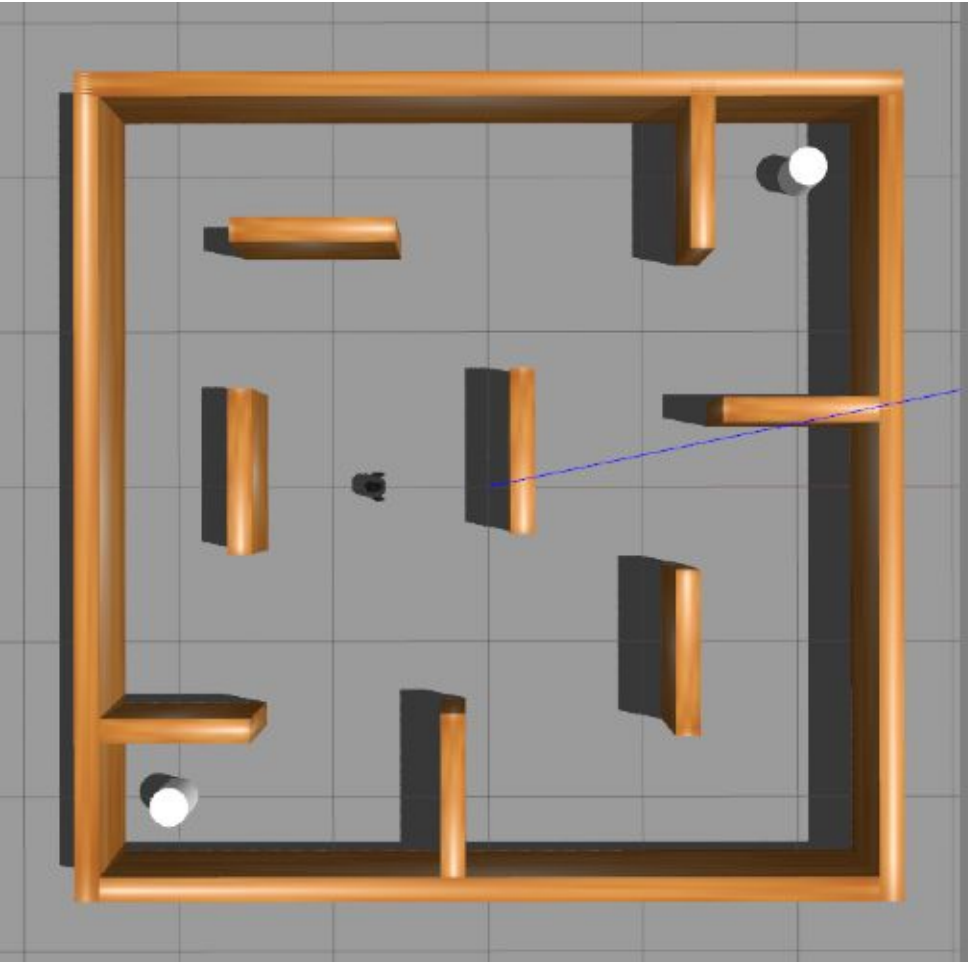
Occupancy grids are used to represent a robot workspace as a discrete grid.

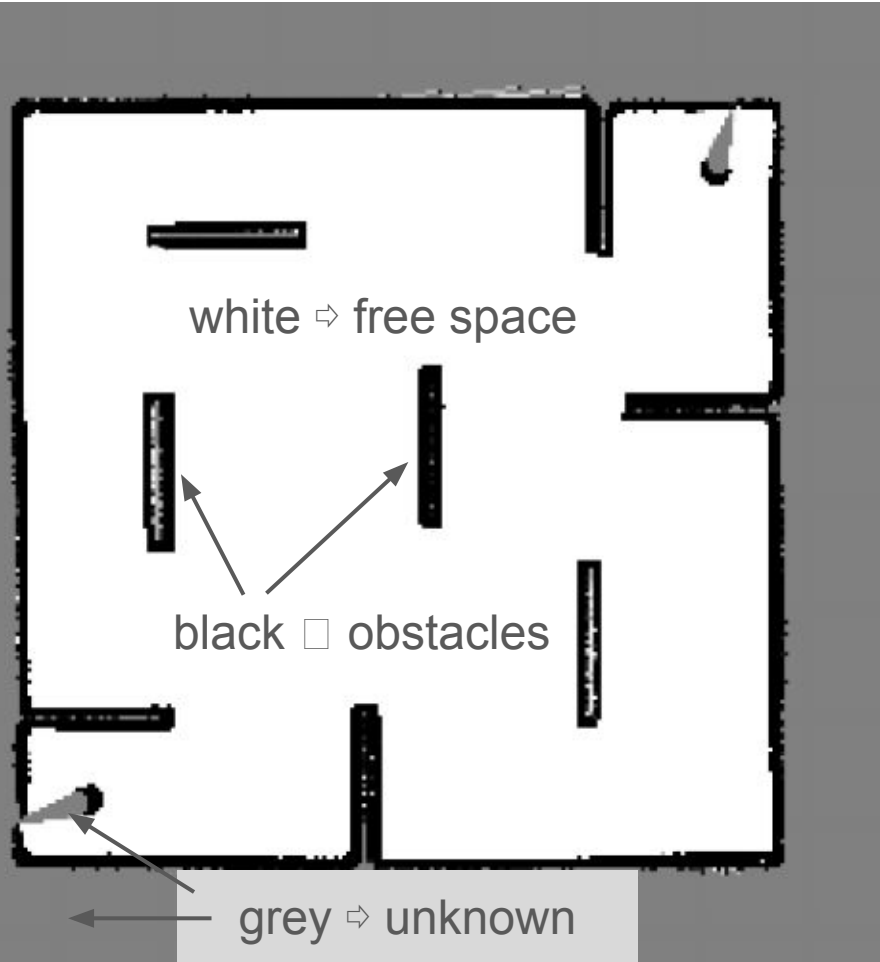
Information about the environment can be collected from sensors in real time or be loaded from prior knowledge.

Laser range finders, bump sensors, cameras, and depth sensors are commonly used to find obstacles in your robot's environment.



Occupancy grid map





SLAM with TurtleBot 3 in Gazebo

Prerequisites:

```
sudo apt install ros-humble-turtlebot3-cartographer
```

Terminal 1:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_house.launch.py
```

SLAM with TurtleBot 3 in Gazebo

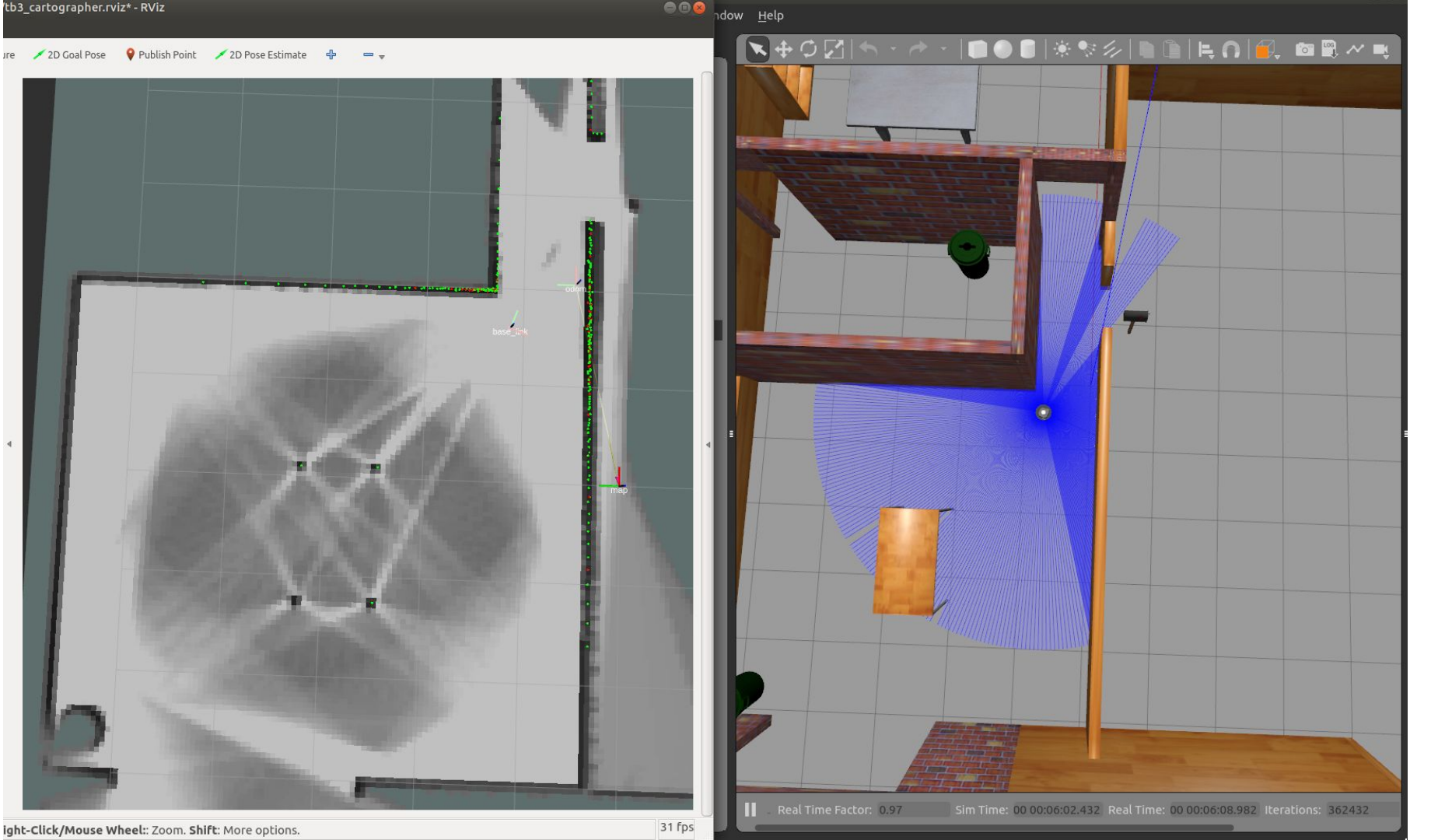
Terminal 2:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 run turtlebot3_teleop teleop_keyboard
```

SLAM with TurtleBot 3 in Gazebo

Terminal 3:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_cartographer \
 cartographer.launch.py use_sim_time:=True
```


tb3_cartographer.rviz* - RViz
2D Goal Pose
Publish Point
2D Pose Estimate
odom
base_link
map
Help
Real Time Factor: 0.97
Sim Time: 00 00:06:02.432
Real Time: 00 00:06:08.982
Iterations: 362432
ight-Click/Mouse Wheel:: Zoom. Shift: More options.
31 fps

Save map

Terminal 4:

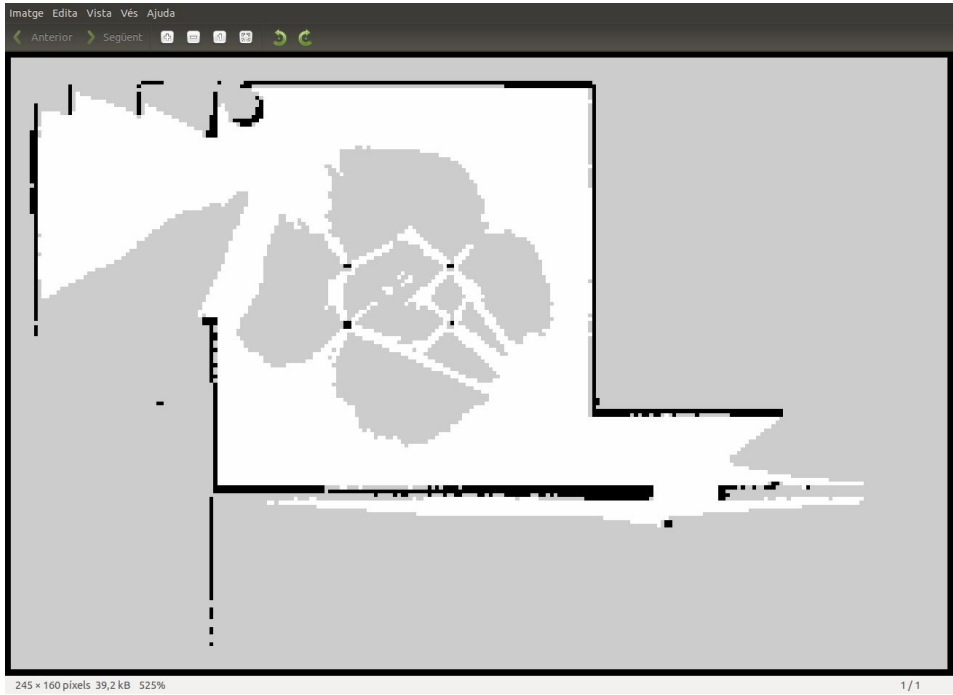
```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
ros2 run nav2_map_server map_saver_cli -f ./map
```

Save map

map.yaml

```
image: ./map.pgm
mode: trinary
resolution: 0.05
origin: [-5.65, -2.02, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

map.pgm

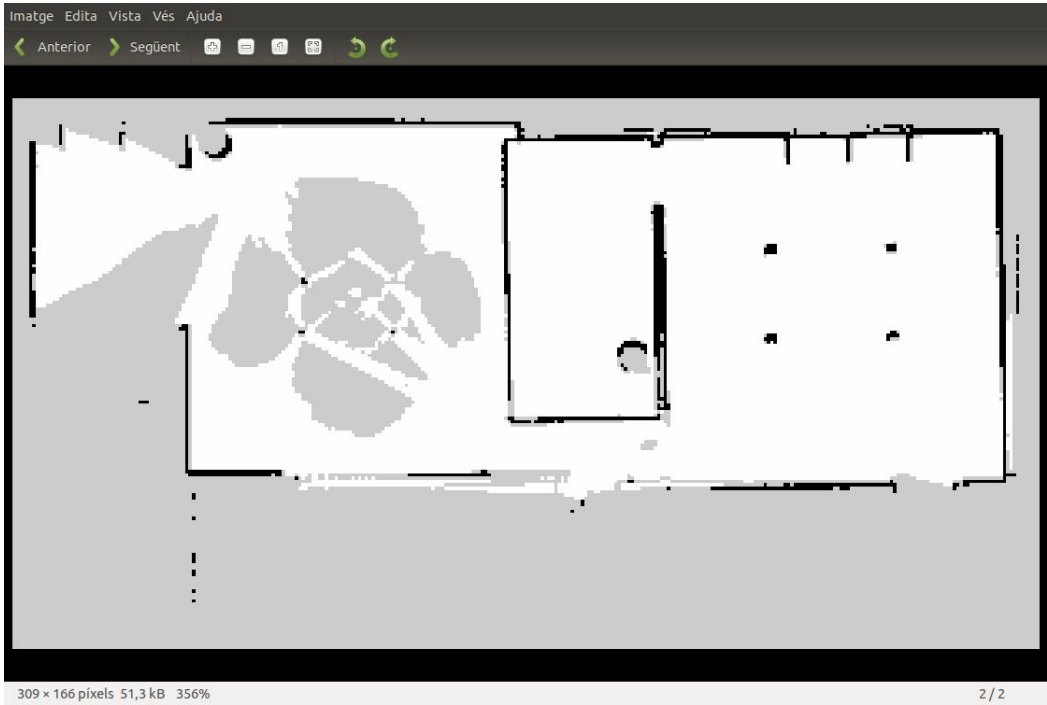


Save new map

map2.yaml

```
image: ./map2.pgm
mode: trinary
resolution: 0.05
origin: [-5.74, -2.44, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

map2.pgm

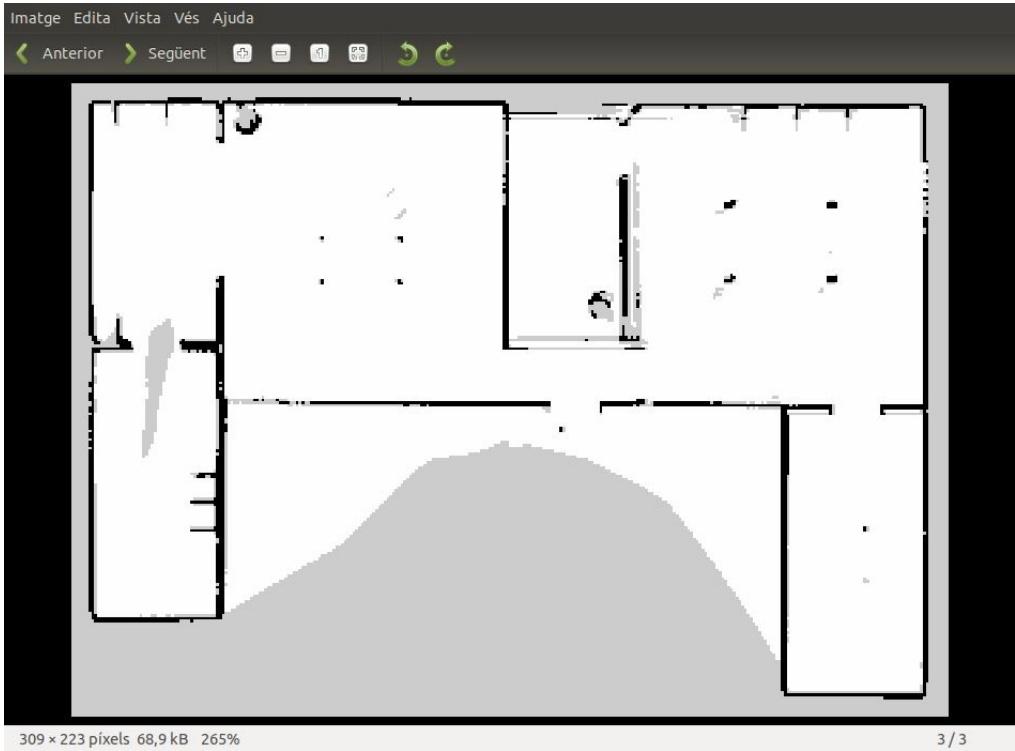


Save new map

map3.yaml

```
image: ./map3.pgm
mode: trinary
resolution: 0.05
origin: [-5.68, -5.2, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

map3.pgm



SLAM in ROS2: Cartographer

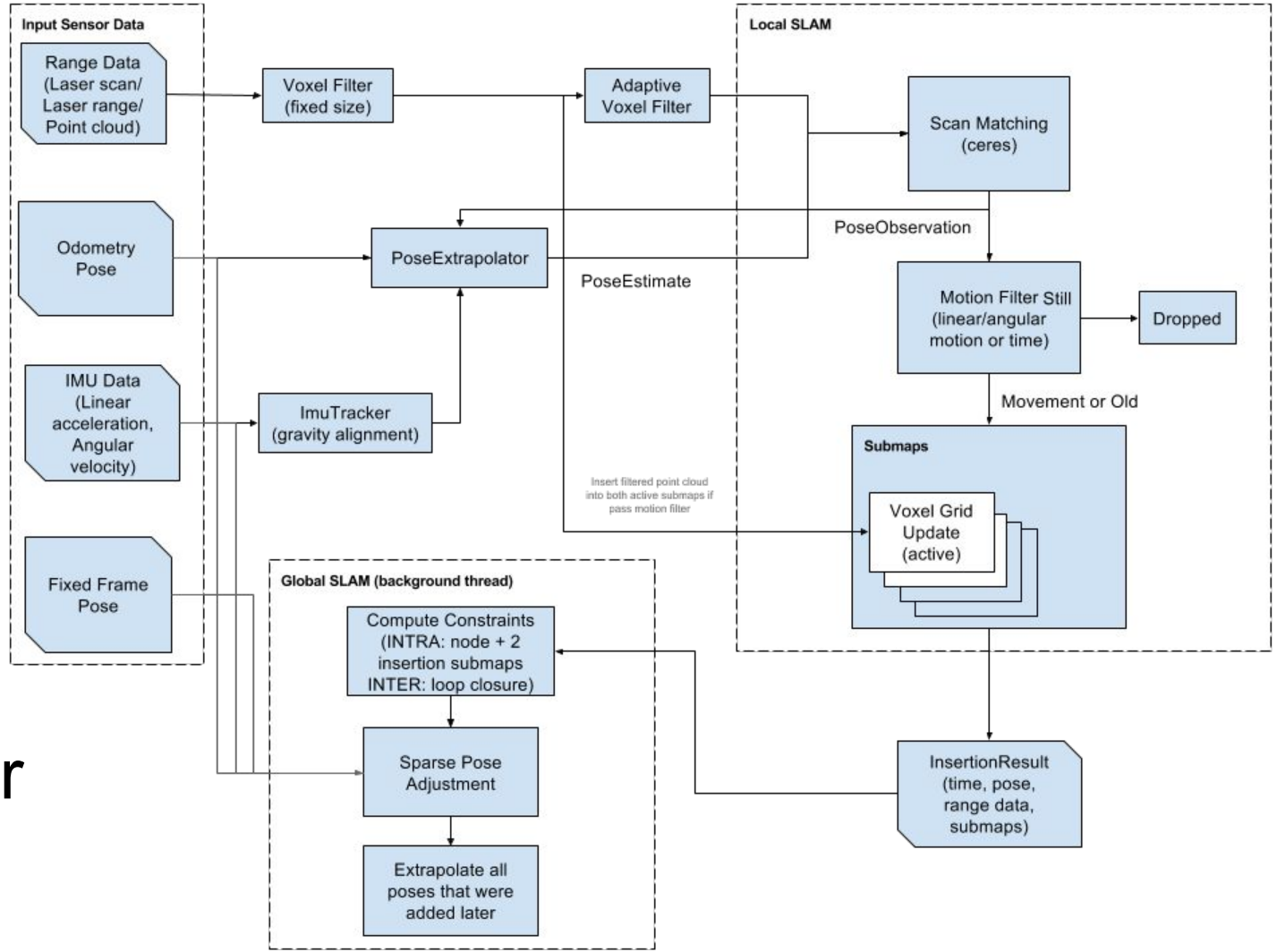
Cartographer is a complex system and tuning it requires a good understanding of its inner working.

If you are interested in more than an introduction to Cartographer, you should refer to the Cartographer paper (below).

It describes the 2D SLAM and defines rigourously most of the concepts described here. Those concepts generally apply to 3D as well.

W. Hess, D. Kohler, H. Rapp, and D. Andor, Real-Time Loop Closure in 2D LIDAR SLAM, in Robotics and Automation (ICRA), 2016 IEEE International Conference on. IEEE, 2016. pp. 1271–1278.

Cartographer Overview



Overview 1

Cartographer can be seen as two separate,
but related subsystems.

The first one is local SLAM
(sometimes also called frontend or local trajectory builder).
Its job is to build a succession of submaps.

Each submap is meant to be locally consistent
but we accept that local SLAM drifts over time.

Overview 2

The other subsystem is global SLAM
(sometimes called the backend).

It runs in background threads and its main job is
to find loop closure constraints.

It does that by scan-matching scans (gathered in nodes)
against submaps.

It also incorporates other sensor data to get a higher level
view and identify the most consistent global solution.

Input

Range finding sensors (e.g. LIDARs) provide **depth information in multiple directions**. However, some of the measurements are irrelevant for SLAM.

If the sensor is partially covered with dust or if it is directed towards a part of the robot, some of the measured distance can be considered as noise for SLAM.

On the other hand, some of the furthest measurements can also come from undesired sources (reflection, sensor noise) and are irrelevant for SLAM as well.

To tackle those issue, Cartographer starts by applying a bandpass filter and only **keeps range values between a certain min and max range**.

Those min and max values should be chosen according to the specifications of your robot and sensors.

Local SLAM

Once a scan has been assembled and filtered from multiple range data, it is ready for the local SLAM algorithm.

Local SLAM inserts a new scan into its current submap construction by scan matching using an initial guess from the pose extrapolator.

The idea behind the pose extrapolator is to use sensor data of other sensors besides the range finder to predict where the next scan should be inserted into the submap.

Global SLAM

While the local SLAM generates its succession of submaps, a global optimization task runs in background.

Its role is to re-arrange submaps between each other so that they form a coherent global map.

For instance, this optimization is in charge of altering the currently built trajectory to properly align submaps with regards to loop closures.

Task

Teleoperate the robot in the Turtlebot 3 Gazebo house environment

Run Cartographer SLAM

Record the following topics in a ROS bag:

```
/clock /map /map_updates /odom /robot_description /scan
  /scan_matched_points2 /submap_list /tf /tf_static
```

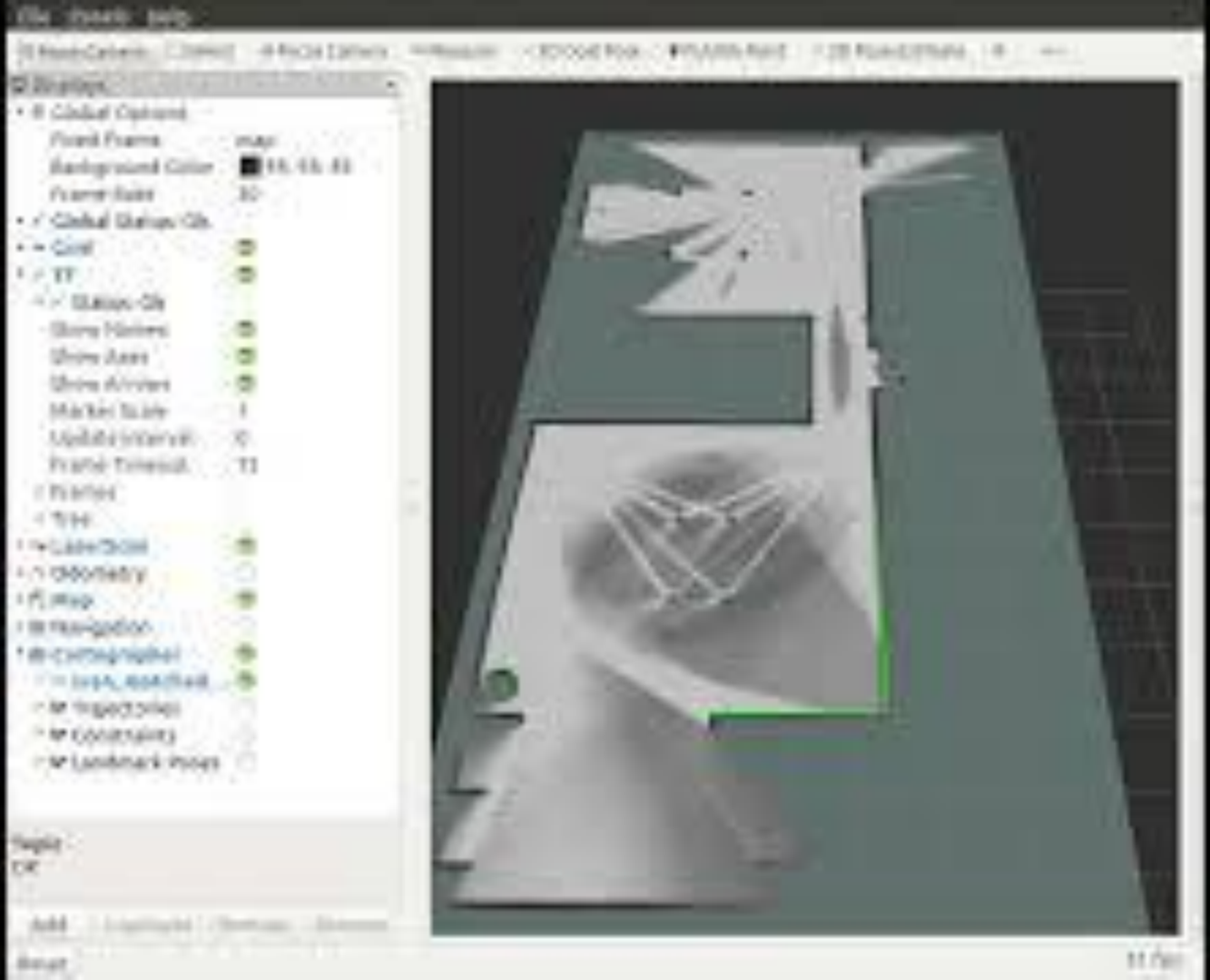
During teleoperation, save a sequence of maps with the map saver utility

Create a PDF document with the sequence of map images

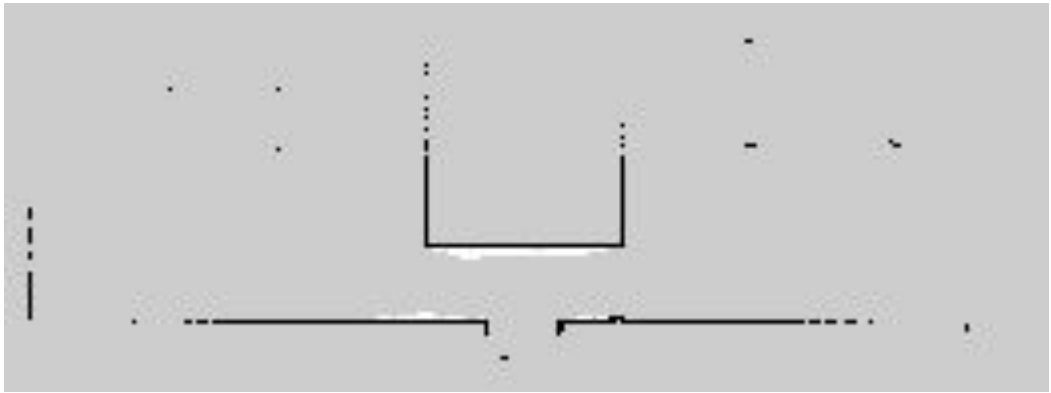
Submit the (zipped) ROS bag file and the PDF document to Aula Virtual

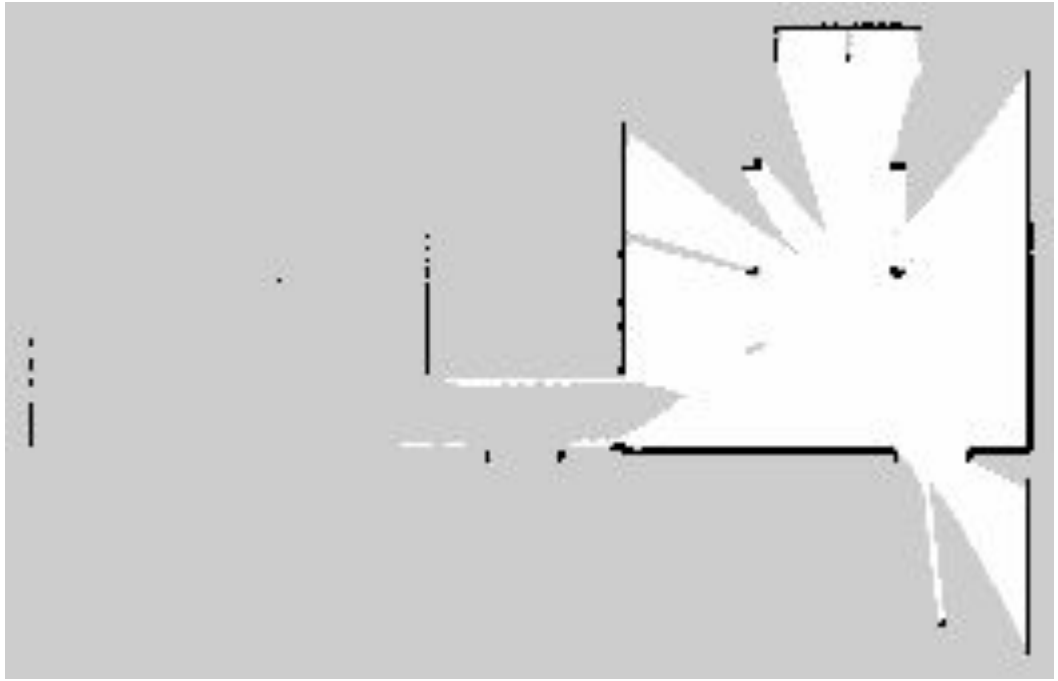
You can verify your bag file with:

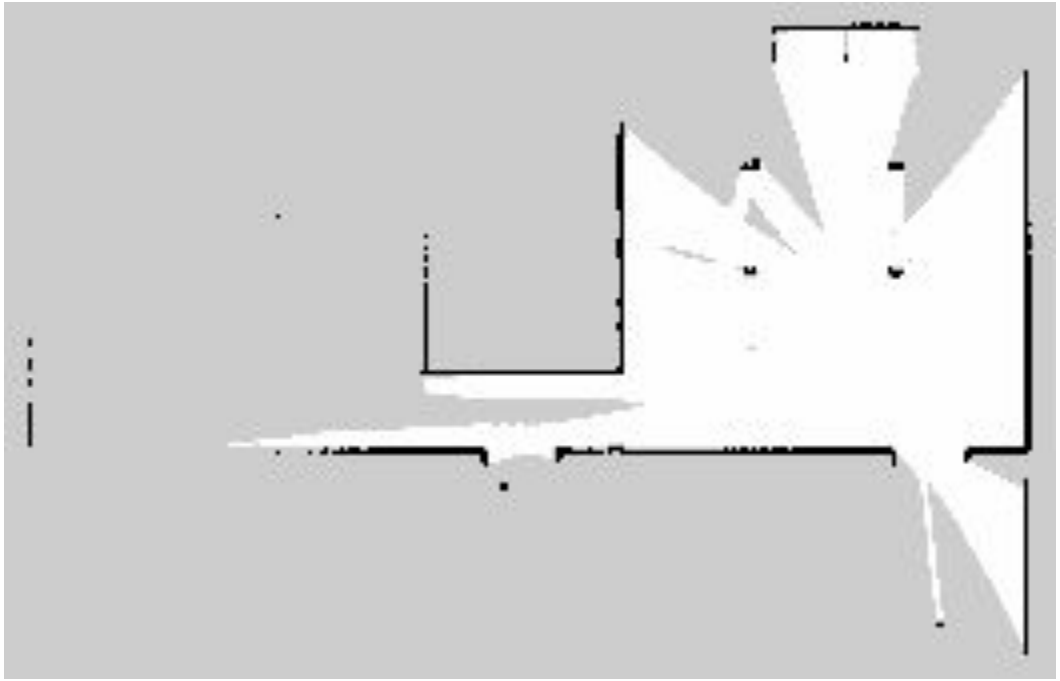
```
rviz2 -d /opt/ros/humble/share/turtlebot3_cartographer/rviz/tb3_cartographer.rviz
```

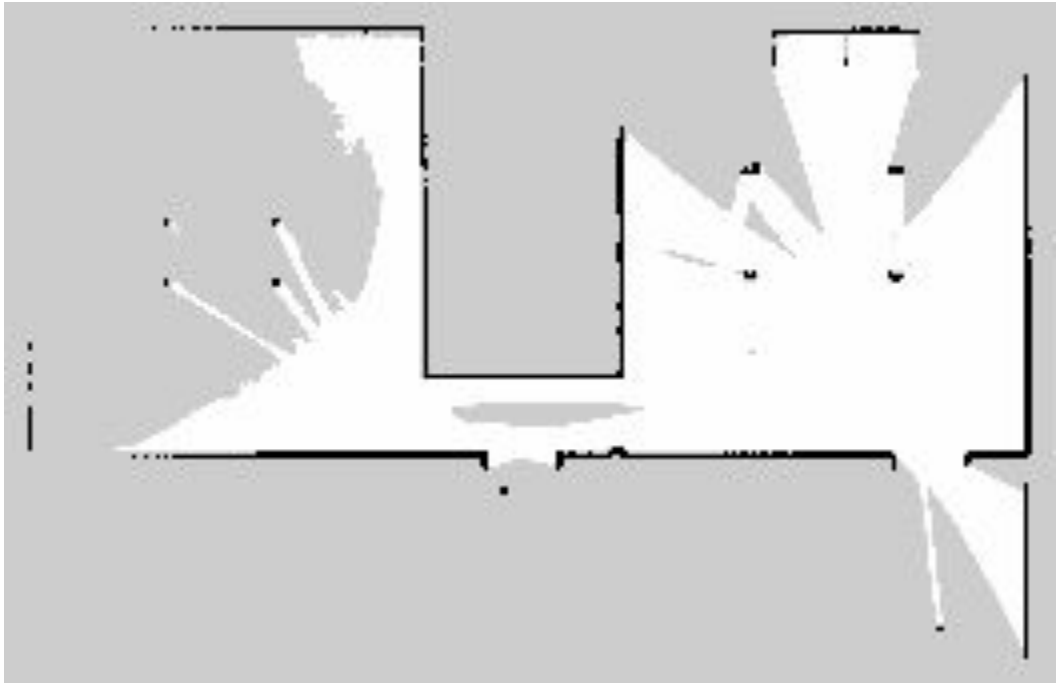


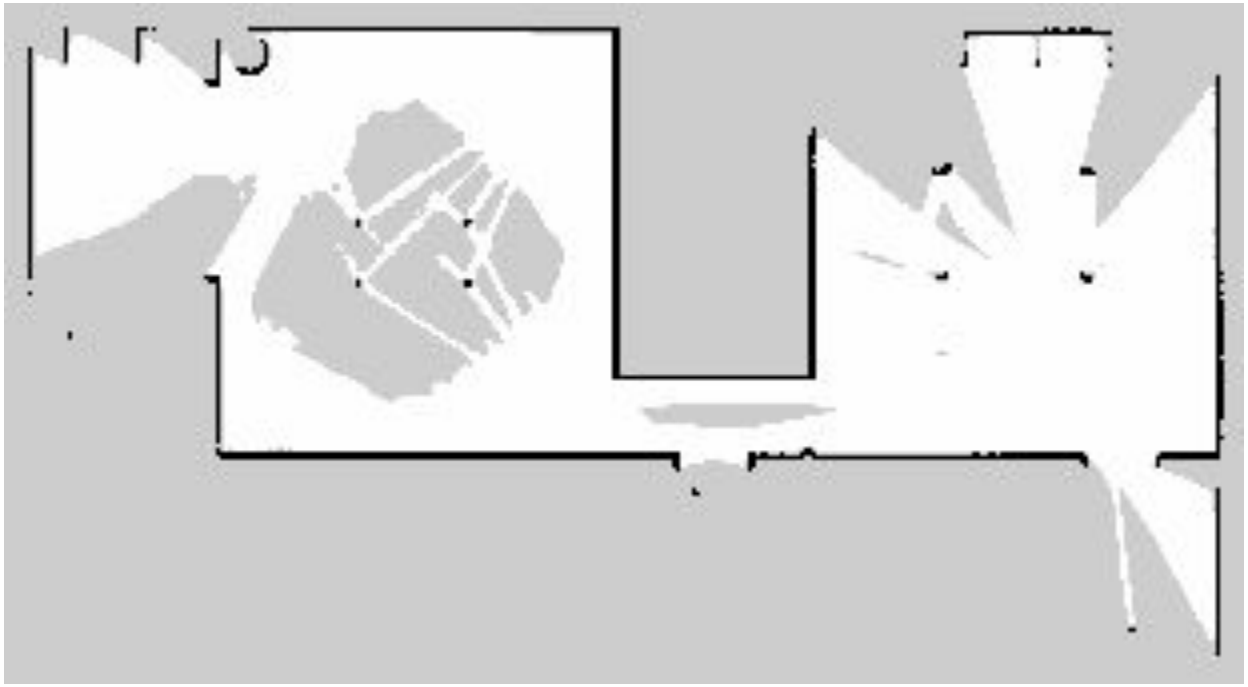


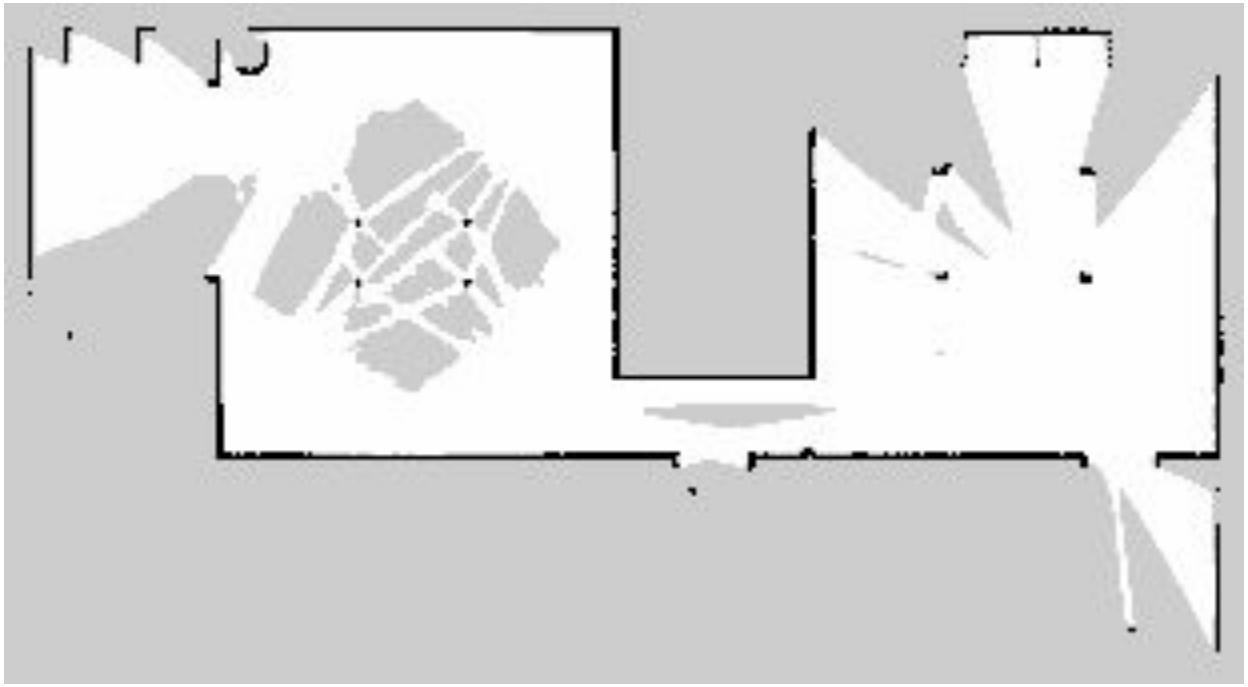


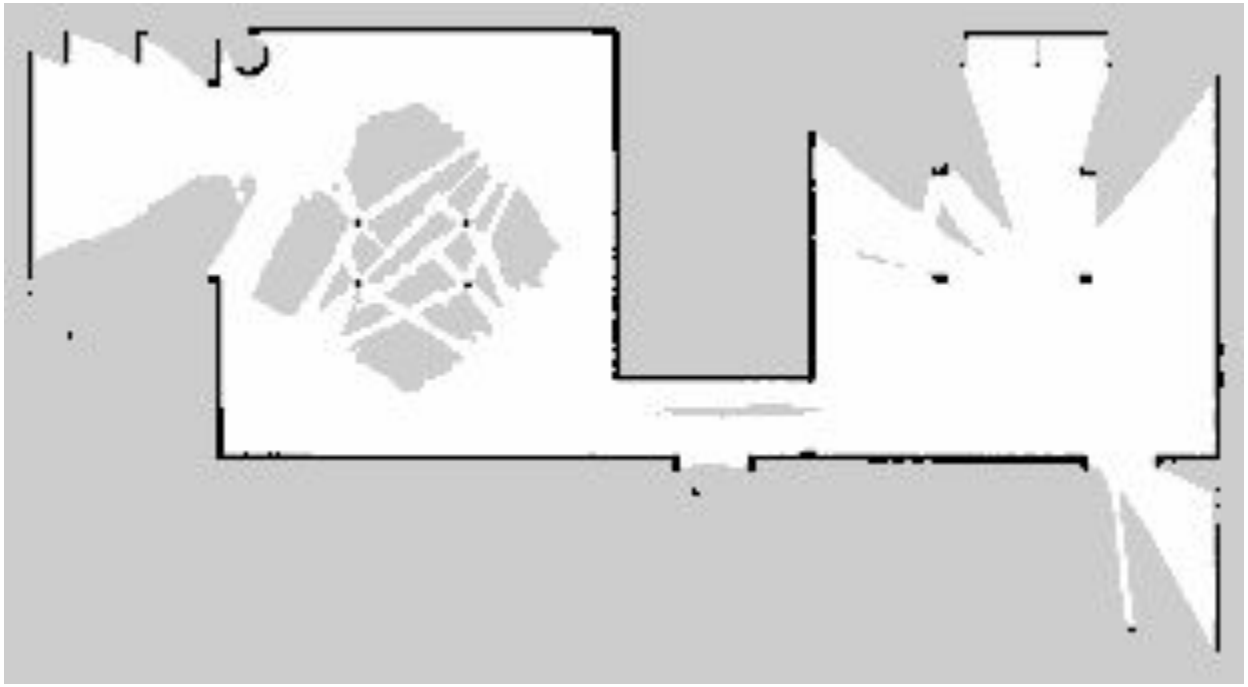


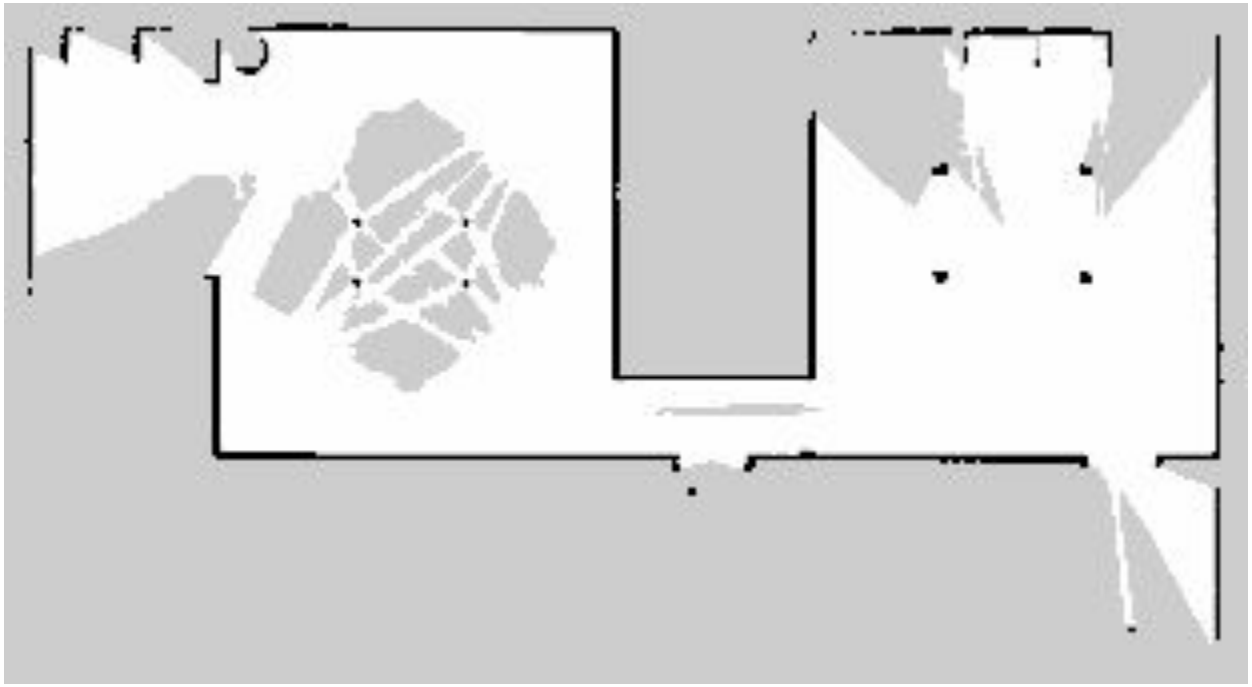












Task

Teleoperate the real Turtlebot 3 robot in the laboratory building

Run Cartographer SLAM

Record the following topics in a ROS bag:

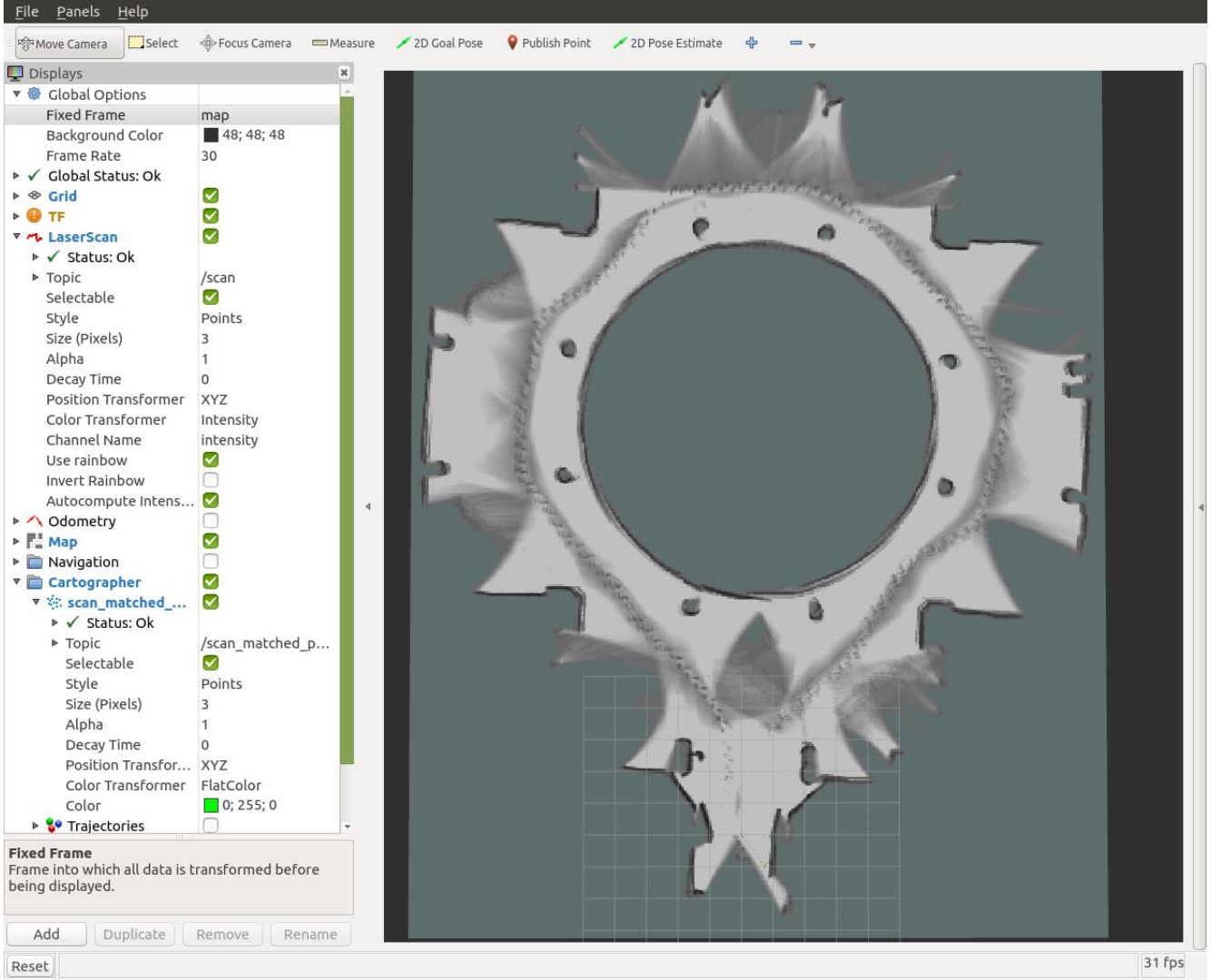
```
/map /map_updates /odom /robot_description /scan
/scan_matched_points2 /submap_list /tf /tf_static
```

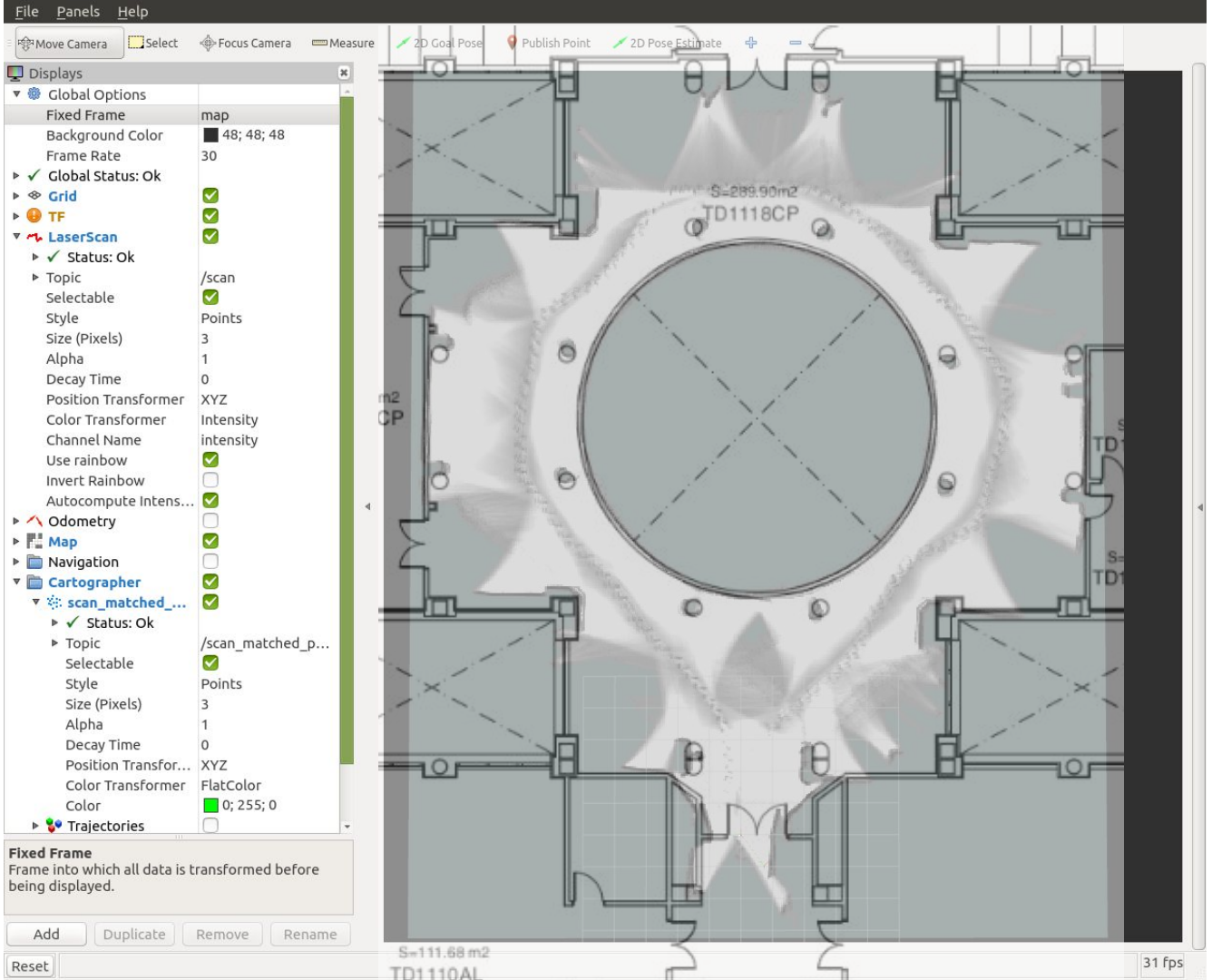
Save a sequence of maps with the map saver utility

Create a PDF document with the sequence of map images

Overlap the last image with the map of the TD building

Submit the ROS bag file and the PDF document to Aula Virtual


File Panels Help
Move Camera
Select
Focus Camera
Measure
2D Goal Pose
Publish Point
2D Pose Estimate
Displays
Global Options
Fixed Frame map
Background Color 48; 48; 48
Frame Rate 30
Global Status: Ok
Grid
TF
LaserScan
Status: Ok
Topic /scan
Selectable
Style Points
Size (Pixels) 3
Alpha 1
Decay Time 0
Position Transformer XYZ
Color Transformer Intensity
Channel Name intensity
Use rainbow
Invert Rainbow
Autocompute Intens…
Odometry
Map
Navigation
Cartographer
scan_matched_…
Status: Ok
Topic /scan_matched_p…
Selectable
Style Points
Size (Pixels) 3
Alpha 1
Decay Time 0
Position Transfor… XYZ
Color Transformer FlatColor
Color 0; 255; 0
Trajectories
Fixed Frame
Frame into which all data is transformed before being displayed.
Add
Duplicate
Remove
Rename
Reset
31 fps


File Panels Help
Move Camera
Select
Focus Camera
Measure
2D Goal Pose
Publish Point
2D Pose Estimate
Displays
Global Options
Fixed Frame map
Background Color 48; 48; 48
Frame Rate 30
Global Status: Ok
Grid
TF
LaserScan
Status: Ok
Topic /scan
Selectable
Style Points
Size (Pixels) 3
Alpha 1
Decay Time 0
Position Transformer XYZ
Color Transformer Intensity
Channel Name intensity
Use rainbow
Invert Rainbow
Autocompute Intens…
Odometry
Map
Navigation
Cartographer
scan_matched_…
Status: Ok
Topic /scan_matched_p…
Selectable
Style Points
Size (Pixels) 3
Alpha 1
Decay Time 0
Position Transfor… XYZ
Color Transformer FlatColor
Color 0; 255; 0
Trajectories
Fixed Frame
Frame into which all data is transformed before being displayed.
Add
Duplicate
Remove
Rename
Reset
S=289.90m2
TD1118CP
S=111.68 m2
TD1110AL
31 fps

Other SLAM algorithms

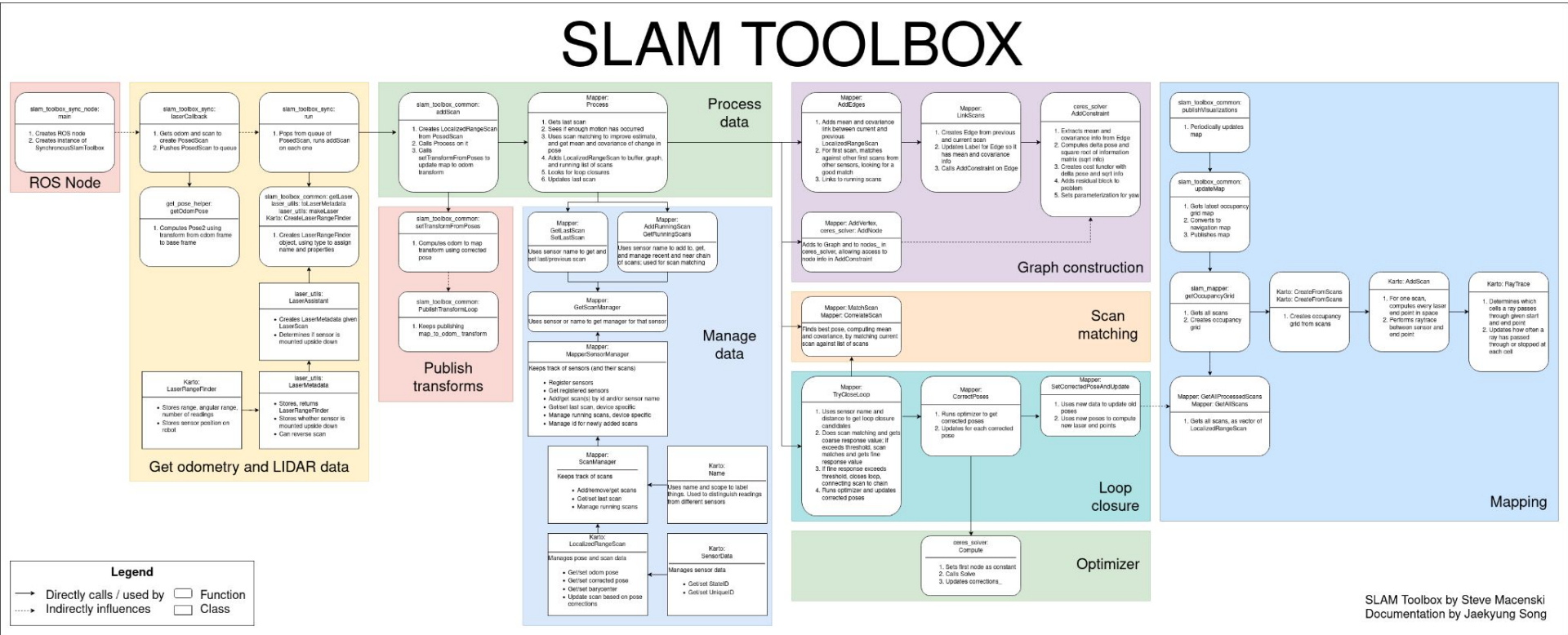
ROS2 SLAM Toolbox

https://github.com/SteveMacenski/slam_toolbox

RTAB-Map

rtabmap_ros/ros2

ROS2 SLAM Toolbox



SLAM Toolbox

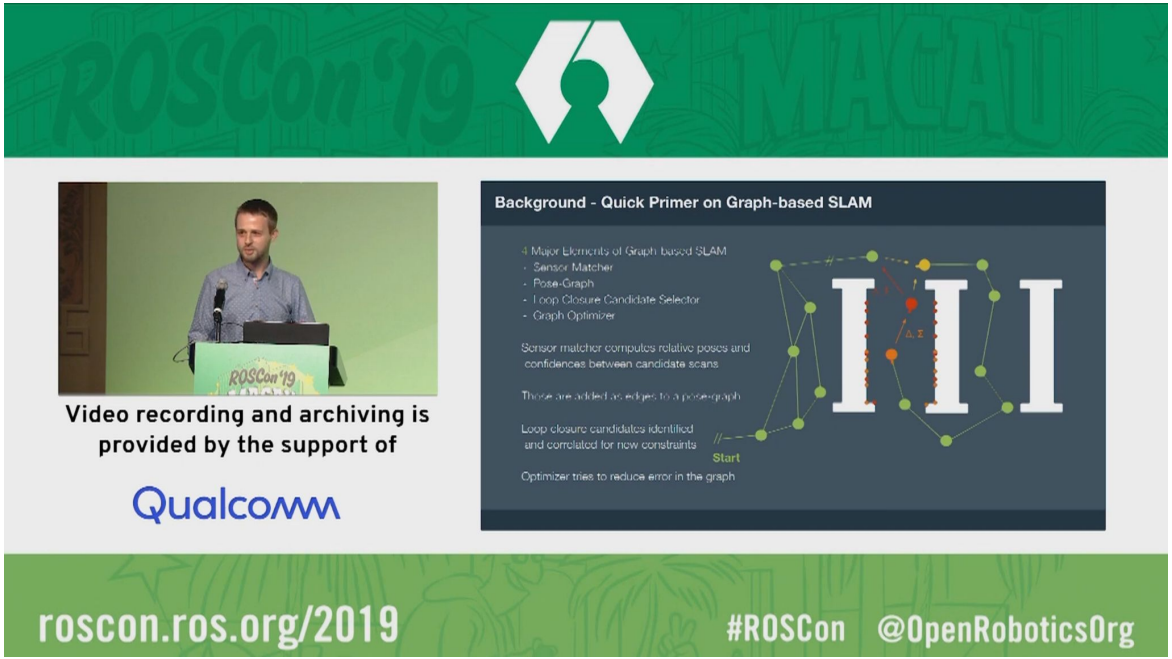
Install:

```
sudo apt install ros-humble-slam-toolbox
```

Run:

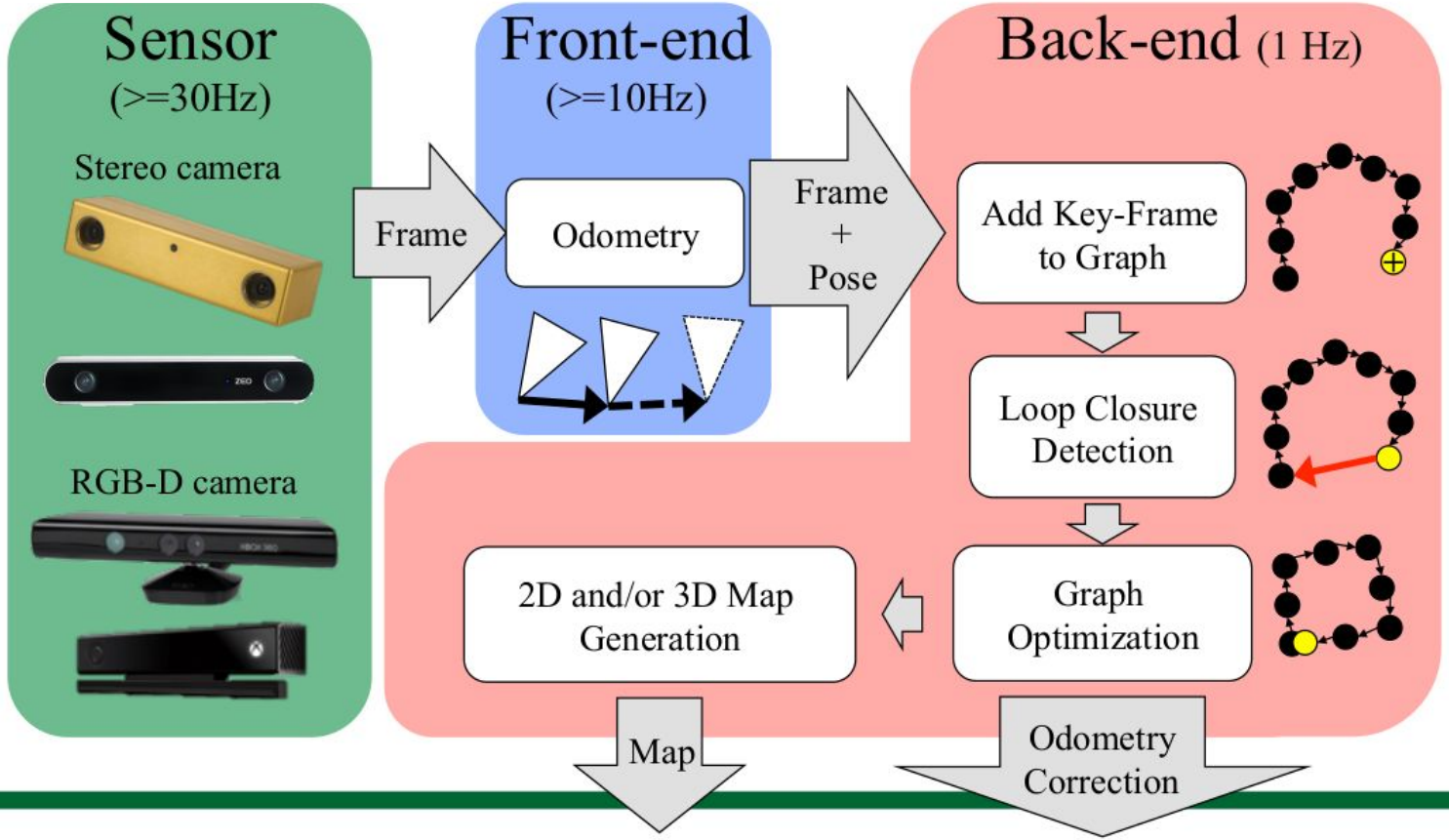
```
ros2 launch slam_toolbox \
  online_async_launch.py \
  use_sim_time:=True
```

SLAM Toolbox



Macenski, S., Jambrecic I., SLAM Toolbox: SLAM for the dynamic world, *Journal of Open Source Software*, 6(61), 2783, 2021.

RTAB-Map: Real-Time Appearance-Based Mapping



RTAB-Map

Install:

```
sudo apt install ros-humble-rtabmap-ros
```

Run:

```
ros2 launch rtabmap_demos \
  turtlebot3_scan.launch.py \
  use_sim_time:=True
```

RTAB-Map



M. Labbé and F. Michaud, “RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation,” in *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.

HOMEWORK: Comparison of SLAM methods

Teleoperate the robot in your Gazebo model of the TD building.

Record the following topics in a ROS bag:

```
/clock /odom /robot_description /scan /tf /tf_static
```

Run each SLAM algorithm (Cartographer, SLAM Toolbox, RTAB-Map) on a play of the bag file and save a sequence of the generated maps.

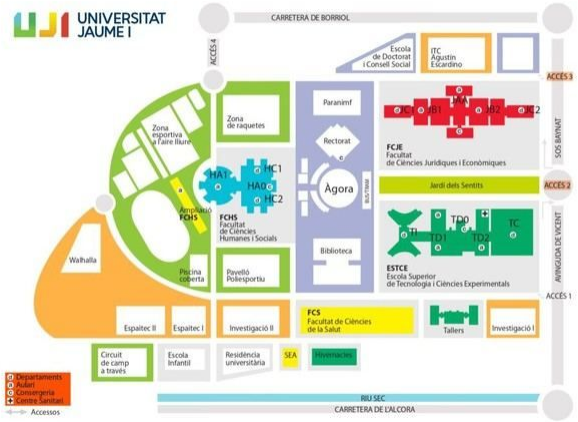
Create a PDF document with the sequence of map images, and add your comments about the result of each algorithm.

Submit the (zipped) ROS bag file and the PDF document.

Mapping the UJI campus

Map one of the floors of a UJI building:

- ESTCE TD: floors 0, 1, 2 or 3, with TurtleBot 2.
- FCJE A: floors soterrani, 0, 1 or 2, with TurtleBot 3.
- FCHS: floors 0, 1, 2 or 3, with TurtleBot 3.
- FCS MD0-MD1-MD2: ground floor with TurtleBot 3.



Work individually or in groups of 2 persons maximum.

Use any SLAM algorithm of your choice.

Submit the (zipped) ROS bag file and a PDF document with a sequence of map images (overlap the last image with the map of the building). Indicate the SLAM algorithm that you have used.